Library Management System

A Console-Based Java Application for Managing Books

NAME- SULAGNA GHOSH

STREAM-IT

ROLL-64

Made with GAMMA

Why This System Was Built

Problem Statement

Traditional library record-keeping using paper logs or spreadsheets is inefficient, error-prone, and difficult to scale. Libraries struggle with:

No Tracking of Book Availability

Librarians cannot instantly know if a book is issued or available.

Manual Issue & Return Records

Paper-based systems lead to data loss and mismanagement.

Scalability Issues

Managing a large collection manually becomes unsustainable.

Project Objectives

Automate Book Management

Add, display, issue, and return books through a simple menu interface.

Apply OOP Concepts

Demonstrate encapsulation, classes, and objects in a real-world context.

Maintain Book Status

Track whether each book is **Available** or **Issued** at any time.

Provide a User-Friendly Interface

Deliver a clear, numbered console menu for all operations.

Made with GAMMA

Who Is This System For?

The Library Management System is designed to serve users across academic and small-scale library environments who need a lightweight, efficient book tracking solution.

School & College Librarians

Manage daily book issue and return operations without complex software.

Students (Borrowers)

Interact with the system to borrow and return books by entering a Book ID.

Java Learners & Developers

Study and understand the practical application of OOP concepts in Java.

System Administrators

Add new books to the library catalog and maintain the collection inventory.

Made with GAMMA

What the System Can Do

The Library Management System provides five key operations accessible through a numbered console menu, covering the complete lifecycle of a library book.

1

Add Book

Register a new book with a unique **ID**, **Title**, and **Author**. Books are stored in an `ArrayList` within the Library.

2

Show All Books

Display the complete book catalog with each book's ID, Title, Author, and current status — **Available** or **Issued**.

3

Issue Book

Mark a book as **Issued** by its ID. The system validates availability and rejects already-issued books.

4

Return Book

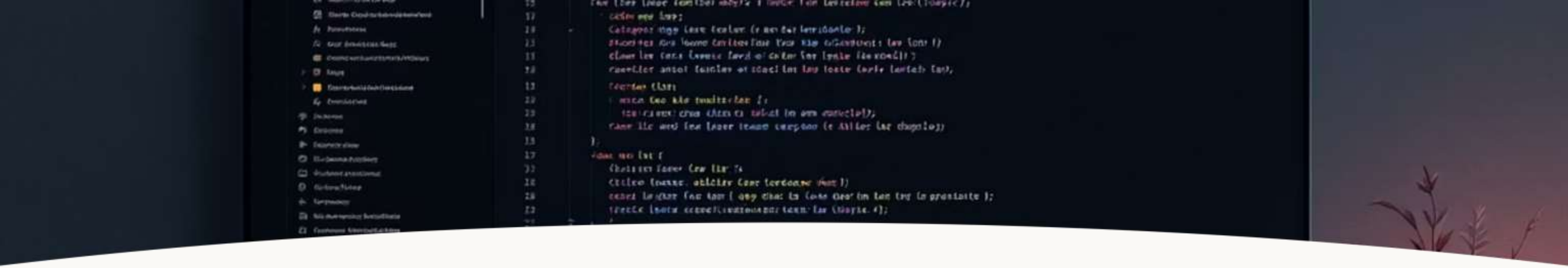
Mark a previously issued book as **Available** again. Validates that the book was actually issued before accepting the return.

5

Exit

Gracefully terminate the program using a menu option, returning control to the operating system.

Made with GAMMA



Object-Oriented Programming Concepts Used

This project is a practical demonstration of core OOP principles applied in Java to model real-world library operations.

Encapsulation

All fields in the `Book` class (`id`, `title`, `author`, `isIssued`) are declared **private**. Public getter and setter methods control access.

Classes & Objects

Two primary classes — `Book` and `Library` — model real-world entities. Each `Book` instance is a distinct object with its own state.

Constructors

The `Book` constructor initializes every new book object with `id`, `title`, `author`, and a default `isIssued = false`.

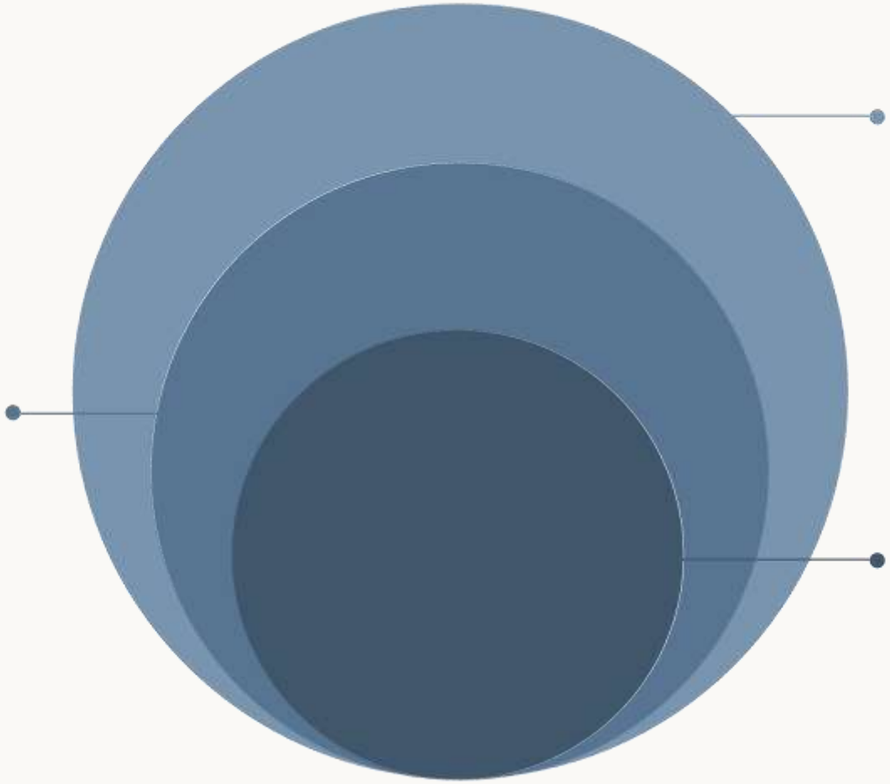
Collections (ArrayList)

The `Library` class uses an `ArrayList<Book>` to store and manage a dynamic collection of `Book` objects at runtime.

Made with GAMMA

System Architecture

The system follows a simple three-layer architecture with clear separation between the data model, business logic, and user interface.



Data Model

Book class: id, title, author, isIssued

Business Logic

Library class: add/show/issue/return

User Interface

Main class, Scanner, console menu

The **Main** class acts as the entry point and UI controller, the **Library** class manages the collection and business rules, and the **Book** class represents the core data entity — each with clearly defined responsibilities.

Made with GAMMA

Code Structure Overview

The entire project is organized into three classes within a single Java file, each responsible for a distinct role in the application.

Class: `Book`

Fields

`private int id` · `private String title` · `private String author` · `private boolean isIssued`

Constructor

`Book(int id, String title, String author)` — sets all fields; `isIssued` defaults to `false`.

Methods

`getId()` · `isIssued()` · `issueBook()` · `returnBook()` · `display()`

Class: `Library`

Field

`private ArrayList<Book> books` — stores the full catalog.

Methods

`addBook(Book)` · `showBooks()` · `issueBook(int id)` · `returnBook(int id)`

Class: `Main` (Entry Point)

```
public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);
  Library lib = new Library();

  while (true) {
    System.out.println(
      "1.Add 2.Show 3.Issue
       4.Return 5.Exit");
    int ch = sc.nextInt();

    switch (ch) {
      case 1: // Add new Book
      case 2: // Show all Books
      case 3: // Issue a Book
      case 4: // Return a Book
      case 5: return; // Exit
      default:
        System.out.println(
          "Invalid choice");
    }
  }
}
```

The `while(true)` loop keeps the menu active until the user explicitly selects **option 5 (Exit)**. Input is handled via `Scanner` and `sc.nextLine()` is used after `nextInt()` to consume the leftover newline character.

Made with GAMMA

System Output & Working

Below is a walkthrough of the console interaction when a user runs the Library Management System, demonstrating each of the five menu operations.

Console Menu & Sample Session

```
1.Add  2.Show  3.Issue  4.Return  5.Exit
> 1
ID: 101
Title: Java Programming
Author: James Gosling
Book added!

1.Add  2.Show  3.Issue  4.Return  5.Exit
> 1
ID: 102
Title: Data Structures
Author: Mark Allen Weiss
Book added!

1.Add  2.Show  3.Issue  4.Return  5.Exit
> 2
101 | Java Programming | James Gosling | Available
102 | Data Structures  | Mark Allen Weiss | Available

1.Add  2.Show  3.Issue  4.Return  5.Exit
> 3
Enter ID: 101
Book issued!

1.Add  2.Show  3.Issue  4.Return  5.Exit
> 2
101 | Java Programming | James Gosling | Issued
102 | Data Structures  | Mark Allen Weiss | Available

1.Add  2.Show  3.Issue  4.Return  5.Exit
> 4
Enter ID: 101
Book returned!

1.Add  2.Show  3.Issue  4.Return  5.Exit
> 5
```

Output Logic Explained

Add Book (Case 1)

Prompts for ID, Title, Author → creates a `new Book()` → appends to `ArrayList` → prints **"Book added!"**

Show Books (Case 2)

Iterates the `ArrayList` → calls `display()` on each `Book` → prints ID, Title, Author, and **Issued/Available** status.

Issue Book (Case 3)

Finds book by ID → checks `!isIssued()` → calls `issueBook()` → sets `isIssued = true` → prints **"Book issued!"** or **"Book not available."**

Return Book (Case 4)

Finds book by ID → checks `isIssued()` → calls `returnBook()` → sets `isIssued = false` → prints **"Book returned!"** or **"Invalid return."**

Made with GAMMA

Conclusion

The **Library Management System** successfully demonstrates how Object-Oriented Programming principles in Java can model and solve a real-world problem in a clean, maintainable way.

OOP in Action

The project puts encapsulation, classes, constructors, and collections to practical use — bridging academic concepts with real implementation.

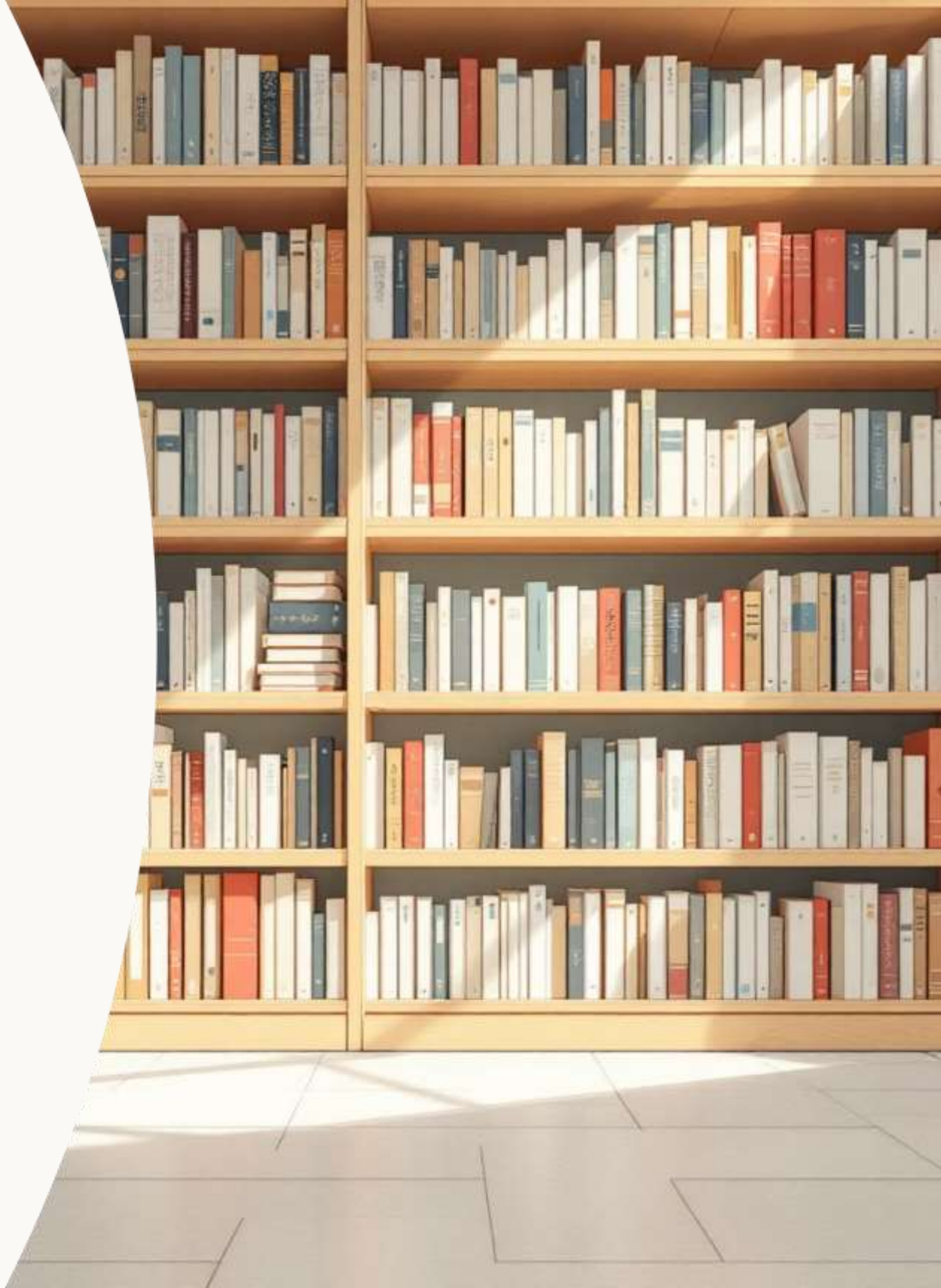
Complete Functionality

All core library operations — adding, viewing, issuing, and returning books — are handled reliably with proper input validation.

Scalable Foundation

The modular class structure makes it straightforward to extend — adding due dates, member profiles, fine calculation, or a GUI in future iterations.

This project lays a strong foundation for more advanced systems — the same OOP architecture can be extended into database-connected or web-based library platforms.



Thank You

Library Management System

We appreciate your time and attention. This project demonstrates a complete, functional Java application built on solid Object-Oriented Programming principles — from a clean data model to a fully interactive console interface.

Language

Java (JDK)

Paradigm

Object-Oriented Programming

Interface

Console / Terminal

Classes

Book · Library · Main